이커머스 서비스 개발

이커머스 설계

서비스 설명

- 온라인에서 상품을 등록하고 판매하는 이커머스 시스템
- 대량의 데이터와 동시접속자가 높은 환경에서도 안정적인 요청 처리
- 판매자와 구매자가 존재하며, 각 역할별 기능 제공

요구사항 정리

- 이커머스는 판매자와 구매자 두 사람이 존재한다
- 판매자
  - 판매자는 이커머스 서비스에 회원 등록해야 한다
  - 판매자는 이커머스 서비스에 로그인해야 한다
  - 판매자는 상품을 등록할 수 있다.
  - 판매자는 상품의 설명, 수량 등 정보를 관리할 수 있다.
- 구매자
  - 구매자는 이커머스 서비스에 회원 등록해야 한다
  - 구매자는 이머머스 서비스에 로그인해야 한다
  - 구매자는 상품을 검색할 수 있다.
  - 구매자는 상품을 검색할 수 있다.
  - 구매자는 배송지를 등록하고 주문시 사용할 수 있다.
  - 구매자는 구매가 완료된 주문의 상태를 조회할 수 있다.
  - 구매자는 구매가 완료된 주문의 배송상태를 조회할 수 있다.
  - 구매자는 주문 내역 리스트를 볼 수 있다.

주요기능

- 1.회원 관리
  - 요구사항
    - 구매자와 판매자는 회원가입을 할 수 있다.
    - 로그인 후 JWT 기반으로 인증을 진행한다.
    - 비밀번호는 해시화하여 저장한다.
    - 회원 정보 조회 및 수정이 가능하다.
  - 데이터 설계
    - `users` 테이블 : 구매자와 판매자 정보를 저장

컬럼이름	데이터 타입	옵션	Null 처리	키설정	설명	
user_id	BIGINT	AUTO_INCREMENT		PRIMARY KEY		
email	VARCHAR(255)		NOT NULL	UNIQUE		

컬럼이름	데이터 타입	옵션	Null 처리	키설정	설명	
password	VARCHAR(255)		NOT NULL			
salt	VARCHAR(255)		NOT NULL			
username	VARCHAR(20)		NOT NULL			
phone_number	VARCHAR(20)		**NOT NULL**	UNIQUE		
role	VARCHAR(20)			NOT NULL	구매자(BUYER) / 판매자(SELLER)	
crea_dt	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP				
updt_dt	TIMESTAMP	DEFAULT CURRENT_TIMESTAMPON UPDATE CURRENT_TIMESTAMP				

- TO-DO
  - 해시? 암호화?
  - 비밀번호 솔트?
  - 토큰의 라이프 사이클을 아는가?
    - 이해를 한후 테이블을 변경하기
  - 상품관리도 유저가 로그인 한 후, 제품정보를 수정하는건가?
    - 어드민 서비스가 필요한가?
    - 현대 이커머스는 판매자용 서비스가 따로 존재한다.
      - 서비스가 중개역할이니, 당근같은 직거래 컨셉보단 이커머스가 적합 > 어드민의 필요성
  - 회원관리부터 기능 개발시작
    - 패키지 디자인까진 비추
    - MVC 형태로.
    - 일관성있게 진행
- 240327
  - userservice
    - 벨리데이션 추가
    - 이메일 이중로그인 체크
      - 에러 잡아서 메세지로 던지기
    - 실무라면 기능 단위로 커밋하기
      - generateSalt >> uuid나 타임스탬프로 문자열처리
    - DDL AUTO 금지 >> 장애유발 가능 >> poc 기능일때만 주로사용
    - password >> 평문 그래도 푸시하지 말자
  - 주석을 달자
  - 첫 통합테스트할때부터 aws인프라에 올리는게 좋을듯.
  - 대칭키는
  - 공개키 / 비공개키
  - 공개키 알고리즘 / 대칭키 알고리즘 언제 사용하는가??
  - 리프레시 토큰이 메인이고, 이를 기반으로 엑세스토큰을 발행한다

        - 엑세스는 수명이 짧고 리프레시는 좀더 길다
        - 예시는 좀 아닌듯.
    - JPA 할줄 안다면, JPA를 정확히 알아야한다
        - jpa할 줄아는데, 장점이 뭔가? 하면 바로 대답이 나와야한다
        - 사용했다면 정확히 알자!
        - 이 기종간의 큰 호환이 큰 장점
        - jpa 는 맵퍼고 내부는 sql
        - jpa보기 어렵다면 ? querydsl 이 이런방식이다.
        - 아는것만 정확히 말하자!!
    - 가능하면 최대한 많이 해봐
        - 설계문서를 준비하자 >>>> 디자인 도큐먼트
        - 깃허브위키
    - 생성자패턴을 지양하려고 빌더패턴이 등장
        - 빌더가 좀더 깔끔
        - 생성자를 요구하는 프레임워크가 있어서 빌더의 제한이 있음
    - 엔티티를 직접다룬다 >> 지양하자!
        - VO 오브젝트로 컨버팅해서 처리하자
        - 엔티티는 트랜잭션단위로 작동시 오류가 발생하기 쉽다
    - 소프트 딜리트로 플래그값으로 처리하는게 더 좋다
        - 컬럼을 벨리드, 딜리트 로 생성
        - 해당 컬럼엔 1,0으로 불린 값으로 처리하는게 좋다
        - 프로덕트 이미지, 카테코리도 소프트 딜리트 값을 넣는게 좋다
    - 이미지 리포지토리는 하나만 사용하자
    - 하나의 서비스 : 하나의 레포지토리
        - 이렇게 안하면 여러군데에서 가져다 사용하니 > 코드가 더러워지고 관리가 안된다...
        - 운영관점에서 레거시화되기 딱 좋음
    - 페이지쿼리
        - 이거 안 먹을거 같다
        - 네이티브쿼리라면, 쿼리 그대로 날라가서 매핑이 된다 > 페이징처리가 없다
        - offset, limit 이 없어서 안 먹을거다
    - 도메인별로 서비스레이어가 좋다
        - 패턴을 뒤죽박죽 쓰지 말자
    - 엔티티 다루는게 좀 어려운 초기단계
    - 엔티티는 조인을 걸텐데, 2개의 로딩이 있는데 >> 트랜잭션널 범위에서 값이 있을수도 있고 , 없을때도 있다
        - 함수콜로 넘어다닐텐데
        - 어떤 함수에서, 데이터가 조인되고 데이터가 나갈때 >> 즉 데이터처리 상태나 동시성 문제를 확인하지 못함
        - 커밋하는 시점을 예측하기 어려움
            - 과거엔 set하면 바로 커밋
    - 도메인 주도 개발
        - 사기다
        - 레거시 기반이 ddd 있는데, 최악이다

        - 왜 힙한 이유가 뭔가? 사기꾼같다
        - 모두가 아는 것이 정통이고 좋다 >> 아이티의 본질은 추상화
        - 혹해서 보일수 있지만 걸러내라
        - 그나물에 그밥
        - 기존방식대비 장점이 아예 없고, 이걸 잘쓴다고 의미가 있을까 >
        - 주체가 도메인, 즉 엔티티로 넘어간건데 장점이 뭔가 있는지 >>> 구조만 바뀐느낌
        - 제대로 아는사람도 없고 테스트도 힘들다 >> 리펙토링도 힘들다
        - 스프링은 빈으로 외부 주입받아서 사용하는 방식 >> 컨디셔널하게 프로퍼티가 주입되는 상황 > 제한 > 인터페이스사용이 어렵다...
        - 공부는 좋다 > 알아서 나쁠건 없다
    - MSA는 개발자영역의 기술인지?
        - 비즈니스 목적이 더 크다
        - 인프라 리소스 효율적으로 사용하는 관점이 50%
        - 개발 바이너리 작다 > 피격범위가 적다
        - 도메인간 고립되는 성격이 강해진다
        - 팀으로 나뉜다면 사일로가 발생한다
        - 통합테스트, 디버깅 하는게 어렵다
        - 팀을 잘게 짜르고 워크로드 조율이 장점 > 팀 조율이 너무 좋음
    - 공개키 대칭키 혼합으로 사용한다
        - HTTPS 공개키이자 + 대칭키로 혼합한다
    - TO-DO:이노디비에서 락을 잡는게 쿼리문의 조건문이나 인덱스 탈때 경합으로 발생
    - 트랜젝션 대기 , 락 이런게 보여야 했는데
    - 병렬성으로 쿼리를 날리고 인덱스 고려했을땐대
    - 동시성 스레드만 늘려서만 성능 개선을 했는데, 테이블 물리설계에서도 검토
    - 로우레벨레인지 컨셉이 mysql의 특성
- @AllArgsConstructor @RequiredArgsConstructor @NoArgsConstructor 차이점 저일
- JPA로 uuid저장할때 16진법으로 변환해서 저장하기 조사
- 인터셉터 > json 파싱 >토큰 > 스레드로컬
    - 스레드로컬 : 스레드내에서만 동작하는 방식
    - 유저 요청건에 대해 기하 급수적인 증가는 안되지만, 충분히 토큰을 스레드로컬에 저장,활용가능
- 공통기능을 다루는게, AOP가 있는데 로그인 기능에는 부적합.
- 내 스타일 : rpc 스타일
- 리소스 중심으로 설계하는걸 rest 스타일 deleteId >>> deleteProduct
- 상품이름 유니크로 한 이유는 무엇일까?
- 서비스 1 : 리포지토리 N으로 계속할건가? 서비스 N : 레포지토리 N방식이 코드 구성이나 관리가 좋지만 안해본 방식이라 어색..
- 서비스 위의 서비스까지 괜찮음 > 유도리 > 정답이 결정된거 아님
- **일관성만 지키자**
- 추가개발 : 상품목록 조회도 페이징 처리가 필요하다. 셀러도 많은 판매 아이템을 보유할수 있다.
- 루트커즈를 찾지못하고 해결못한케이스같다.
    - 왜 레프트 조인이 필요한가?
    - 테이블 설계나 구조
    - 인덱스가 잘 걸려있는지, 잘 타는지

    - 역정규화 , 캐싱도 가능하지 않을까
    - 이것도 안되면 어플리케이션성격까지 분석해서 최적화가 필요
-

25/03/13 피드백

- 주문생성,취소 > 카프카발행 > 바로 성공처리로 된다 > 유저는 성공상태값만 본다..
- 데이터소스에서 이벤트가 퍼져나가는거
- 고객인터페이스는 어떻게 필요한가 ? 전체적으로 필요하다...
- 이벤트단위가 아니라, 사용자 관점에서 상태값이 나와야한다
    - 예시
        - 주문처리중
        - 주문추적
        - 주문생성 중
        - 주문취소 중
    - 이벤트는 백앤드만
    - 사용자 관점으로 주문 플로우로 추적하는 인터페이스가 필요하다 > 신규 기능 개발하기
    - dto 재사용이 아니면, 클래스 네이밍이 더 명확하게 하기
    - 로그인을 공통처리하는 필터, 인터셉터 기능이 추가되어야 한다
    - cascade 지양하기. > 실무에서 안 사용함. > 제어가 불가능한 자동화는 사용하기
    - 오더 기능은 페이징 처리 필요
- protobuf는 grpc에서 사용을 많이 한다
    - 전용 포멧터로 생성하고
    - 모델별로 클래스를 새로 만들어라 > 직렬화 클래스 따로 / 역직렬화 클래스 따로....
- 카프카의 토픽, 키 설계
    - 베스트케이스 : 용도에 맞춰서 발행,소비가 좋지...
- JPA 활용법
    - 부모자식 관계
        - 레이지 로딩을 주로
        - 이거 로딩을 가끔하기도함
        - fetch join 도 주로 활용함
        - 예상질문 : n+1 문제는 어떻게 해결했어?
- 이벤트 드리븐 아키텍쳐지향
    - 기능구현보단 EDA를 하고 싶어서 탄생된것
    - 카프카는 응답이 빠르지 않다
        - 제대로 된 응답을 위해 polling 이 필요하다
    - EDA로 카프카를 도입한거라면, 장단점을 명확히 해야 한다
        - 오늘 설명한 내용으론 부족하다
    - 이커머스에선 결제에 대해 알아야 한다
        - 지금했던건 현실상 아님
        - 기본 개념 : 원장 / 영수증
            - 영수증 : 요청에 대한 정보를 쌓기
            - 결제가 트리가될때 영수증이 발행

            - 원장 :
                - 페이먼트는 모두 이벤트로 기록해야 한다
                - 취소, 실패가 다 적용되어야 한다
                - 모두 다 추적이 가능해야 한다.
            - 실제 결제 전에는 인증 단계가이 있다.
                - 현재는 팬딩단계 쯤
        - 재고
            - 결제 성공 재고차감
            - 결제실패하면 재고살리기
        - 어플리케이션 서비스 자체 실패
            - 널포인트,
            - 카프카론 널 잡기 힘듬
        - 이벤트 드리븐 아키텍쳐
            - eda에선 리드온리는 피해야 한다
                - 리드 트랜잭션
                - 라이트 트랜잭션
                    - 서로의 뷰가 다르다
                - 격리레벨 공부하기
                - 트랜잭션걸면 이벤트 다 깨진다, 데이터가 하나로 통일되게 하자.
            - 주문 취소가 먼저 접수되는 이벤트상황
- rdb의 핵심요소
- no-sql 의 헥심이론
    - 분산환경
        - cap 이론
        - acid
- DB단의 동시성처리 락은 디비 사용 코스트가 크니, 레디스의 분산락을 추천
- JPA Dirty Checking
- 트랜젝션 선언한 메서드는 왜 protected만 접근제어 되는지?
- 순환참조 문제로 ApplicationEventPublisher 사용 > 클래스 단위의 이벤트핸들링
    - **Kafka**: 토픽 이름 기반 라우팅
    - **Spring Events**: 클래스 타입 기반 라우팅
- Spring이 이벤트를 매핑하는 방식:
    - **파라미터 타입 기반**: 이벤트 객체의 클래스 타입으로 매칭
    - **상속 관계 지원**: 부모 클래스 이벤트도 받을 수 있음
    - **조건부 처리**: `@EventListener(condition = "...")` 으로 세밀한 제어 가능

  차이점:
    - `ApplicationEventPublisher`: **이벤트 기반 통신** (발행자-구독자 패턴)
    - `@Async`: **단순 비동기 실행** (메서드를 다른 스레드에서 실행)
- 비관적 락의 DB 코스트 분석
    - "DB 코스트"의 실제 의미
    - 1. 커넥션 풀 고갈 : DB 커넥션 1개가 전체 처리 시간 동안 점유됨

- rdb 싱글라이터 > 수평확장이 어렵다 > rdb리소스사용 증가 > rdb는 정합성 케이스에 매우 탁월
- 레디스는 CAD관점에서 수평확장이 가능하다. > rdb와 같이 결합하면 매우 좋다
-

// 비관적 락 사용 시

```
@Transactional
public void deductStocks(List<StockDeductionRequest> requests) {
  // DB 커넥션 1개가 전체 처리 시간 동안 점유됨
  List<Product> products = findByProductUUIDsWithLock(productUUIDs); // 락 획득까지 대기

  for (Product product : products) {
      product.deductStock(quantity);     // 비즈니스 로직 처리
      // 외부 API 호출이나 복잡한 계산이 있다면?
      Thread.sleep(100);                 // 커넥션이 계속 점유됨
  }
  // 여기서야 커넥션 반납
}
```

2. 대기 시간 증가

```
동시 주문 100개가 같은 상품 포함 시:

비관적 락: 99개가 1개 완료까지 대기
- 평균 대기시간: 처리시간 × 순서 = 100ms × 50 = 5초
- 타임아웃 리스크: 일부 요청이 30초 후 실패

Redis 분산락: 락 획득/해제가 빠름
- 평균 대기시간: 5ms × 50 = 250ms
- 락 처리 최적화 가능
```

3. 대기 시간 증가

```
-- MySQL InnoDB의 데드락 감지
-- 100개 트랜잭션이 복잡한 락 체인을 형성할 때
-- 데드락 감지 알고리즘이 CPU를 많이 사용

SHOW ENGINE INNODB STATUS;
-- 데드락 감지: 0.1초마다 스캔 (innodb_deadlock_detect=ON)
-- 복잡도: O(N²) where N = 대기 중인 트랜잭션 수
```

배치(Batch) vs 벌크(Bulk) 처리 차이

배치작업은 실시간성이 아닌 개념

배치 처리 (Batch Processing)

```
// 순차적으로 하나씩 처리하는 방식
public void deductStocks(List<StockDeductRequest> requests, UUID orderUUID) {
    for (StockDeductRequest request : requests) {
        // 각 상품을 개별적으로 순차 처리
        StockDeductResult result = deductStock(request.getProductUUID(), request.getQuantity(), orderUUID);
        results.add(result);
```

```
    }
}
```

실제 DB 쿼리 - 배치 처리 방식

```
-- 각 상품마다 개별 쿼리
SELECT * FROM products WHERE product_uuid = 'product-1' FOR UPDATE;
UPDATE products SET stock_quantity = stock_quantity - 2 WHERE product_uuid = 'product-1';

SELECT * FROM products WHERE product_uuid = 'product-2' FOR UPDATE;
UPDATE products SET stock_quantity = stock_quantity - 1 WHERE product_uuid = 'product-2';

-- 총 N개 상품 × 2번 쿼리 = 2N번 DB 호출
```

벌크 처리 (Bulk Processing)

```
// 한 번의 DB 쿼리로 여러 건을 동시 처리
public void deductStocks(List<StockDeductRequest> requests, UUID orderUUID) {
    // 🔑 한 번에 모든 상품 조회
    List<Product> products = productRepository.findByProductUUIDsWithLock(productUUIDs);

    // 🔑 메모리에서 일괄 처리
    for (StockDeductRequest request : requests) {
        Product product = productMap.get(request.getProductUUID());
        product.deductStock(request.getQuantity());
    }
    // 🔑 트랜잭션 끝에서 한 번에 DB 반영 (JPA Dirty Checking)
}
```

벌크 처리 (Bulk Processing)

```
-- 한 번에 모든 상품 조회
SELECT * FROM products
WHERE product_uuid IN ('product-1', 'product-2', 'product-3')
ORDER BY product_uuid FOR UPDATE;

-- JPA가 변경된 엔티티들을 자동으로 일괄 업데이트
UPDATE products SET stock_quantity = 10 WHERE id = 1;
UPDATE products SET stock_quantity = 15 WHERE id = 2;
UPDATE products SET stock_quantity = 8 WHERE id = 3;

-- 총 1번 조회 + N번 업데이트 (하지만 같은 트랜잭션 내)
```

벌크 처리의 장점

```
// ✅ 현재 구현: All or Nothing 보장
@Transactional
public void deductStocks(...) {
    // 1. 모든 상품 한 번에 락 획득
    // 2. 모든 재고 검증
    // 3. 모든 차감 처리
    // 4. 하나라도 실패 시 전체 롤백

    if (!hasAllsuccess) {
```

```
            throw new RuntimeException(); // 전체 트랜잭션 롤백
        }
    }
```

🎯 결론

1. **용어**: 현재 코드는 "벌크 처리"가 정확한 표현
2. **차감 vs 복구**: 비즈니스 특성에 따른 설계 차이
   - **차감**: 높은 동시성 + 강한 원자성 → 벌크 처리
   - **복구**: 낮은 동시성 + 부분 실패 허용 → 개별 처리
3. **동시성**: 복구도 동시성 이슈 있지만, 현재 락 사용으로 보호됨
4. **개선 여지**: 복구도 벌크 처리로 성능 향상 가능

현재 구현도 충분히 안전하지만, 성능이 중요해지면 복구도 벌크 처리로 개선하는 것을 고려해볼 수 있습니다!

- 2. 상품관리
  - 요구사항
    - 판매자는 상품을 등록, 수정, 삭제할 수 있다.
      - 상품정보는 사진과 설명이 포함된다
    - 구매자는 상품을 검색하고 조회할 수 있다.
      - 카테고리 , 가격, 키워드로 검색 가능해야 한다.
  - `products` 테이블

컬럼이름	데이터 타입	옵션	Null 처리	키설정	설명
id	BIGINT	AUTO_INCREMENT		PRIMARY KEY	row고유번호
seller_id	BIGINT		NOT NULL		판매자 ID
name	VARCHAR(255)		NOT NULL		상품명
description	TEXT				상품 설명
price	DECIMAL(10,2)		NOT NULL		가격
stock_quantity	INT		NOT NULL		재고 수량 —> 수량값으로 완판여부 확인가능
status	VARCHAR(10)	DEFAULT 'ACTIVE’			'ACTIVE', 'HIDDEN','DELETED’
crea_dt	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP			생성일
updt_dt	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP			수정일

  - `product_categories` 테이블 (카테고리 정보)

컬럼이름	데이터 타입	옵션	Null 처리	키설정	설명
id	BIGINT	AUTO_INCREMENT		PRIMARY KEY	상품 ID
name	VARCHAR(255)	UNIQUE	NOT NULL		카테고리 이름 (예: 의류, 전자제품 등)
crea_dt	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	NOT NULL		

- `product_images` 테이블 (상품 이미지 정보)

컬럼이름	데이터 타입	옵션	Null 처리	키설정	설명
id	BIGINT	AUTO_INCREMENT		PRIMARY KEY	
product_id	BIGINT		NOT NULL		상품 ID
image_url	VARCHAR(500)		NOT NULL		이미지 저장 URL
crea_dt	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP			

api

- 상품등록 POST : /product/register/{prouctId}
  - 요청값 : {
    name : ”productName”
    ,description: “description”
    ,price: “21.10"
    ,stockQuantity: “100”
    ,status: “active” }
  - 응답값 : {sellerId : “1234”
    ,name : ”productName”
    ,description: “description”
    ,price: “21.10"
    ,stockQuantity: “100”
    ,status: “active”
    ,imagUrl : “https:aws.s3.com”
    ,createDate: “2025-04-01”
    ,updateDate: “2025-04-01”}
- 상품수정 POST : /product/modify/{prouctId}
  -
    - 요청값 : {
      name : “changeName”
      ,description: “changeDescription”
      ,price: “150.13"
      ,stockQuantity: “10”
      ,imagUrl : “https:azure.s3.com” }
    - 응답값 : {sellerId : ”1234”
      ,name : ”productName”
      ,description: “description”
      ,price: “21.10"
      ,stockQuantity: “100”
      ,status: “active”
      ,createDate: “2025-04-01”
      ,updateDate: “2025-04-01”}
- 상품삭제 POST : /product/delete/{prouctId}
  - 응답값 : {response: “OK”}
- 상품조회 GET. : /productt/search/{prouctId}
  - 요청값 : {
    name : ”changeName”
    ,description: “changeDescription”
    ,price: “150.13"}
  - 응답값 : {sellerId : “1234”
    ,name : ”productName”
    ,description: “description”
    ,price: “21.10"

,stockQuantity: “100”
,status: “active”
,createDate: “2025-04-01”
,updateDate: “2025-04-01”}

3. 주문 & 결제 관리(매우 중요)

- 요구사항
  - 구매자는 장바구니에서 상품을 주문할 수 있다.
  - 결제 후 주문 상태는 자동으로 업데이트된다.
  - 주문 내역 및 상태를 조회할 수 있다.
- `orders` 테이블 (주문 정보)

컬럼이름	데이터 타입	옵션	Null 처리	키설정	설명
id	BIGINT	AUTO_INCREMENT		PRIMARY KEY	내부용 PK
buyer_id	BIGINT		NOT NULL	FOREIGN KEY	구매자 ID
total_price	DECIMAL(10,2)		NOT NULL		
status	VARCHAR(20)	DEFAULT 'PENDING’			'PENDING', 'PAID', 'CANCELLED', 'SHIPPED', 'DELIVERED’
crea_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP			

- `order_items` 테이블 (주문 상품 정보)

컬럼이름	데이터 타입	옵션	Null 처리	키설정	설명
id	BIGINT	AUTO_INCREMENT		PRIMARY KEY	
order_id	BIGINT		NOT NULL	~~FOREIGN KEY REFERENCES orders(id) ON DELETE CASCADE~~	주문 ID TO-DO >> 반정규화가 필요 >> 제품정보를 조인하면 성능 문제 유발 >> 데이터 조인시 어플리케이션에 부담예상됨
product_id	DECIMAL(10,2)		NOT NULL	~~FOREIGN KEY REFERENCES products(id) ON DELETE CASCADE~~	상품 ID
quantity	INT		NOT NULL		구매 수량
price	DECIMAL(10,2)			NOT NULL	개별 상품 가격 (주문 시점 기준)
crea_dt	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP			

- `payments` 테이블 (결제 정보)

컬럼이름	데이터 타입	옵션	Null 처리	키설정	설명
id	BIGINT	AUTO_INCREMENT		PRIMARY KEY	
order_id	BIGINT		NOT NULL	~~FOREIGN KEY REFERENCES orders(id) ON DELETE CASCADE~~	주문 ID
~~payment_method~~	VARCHAR(50)		NOT NULL		~~신용카드, 계좌이체 등 결제 방식~~

컬럼이름	데이터 타입	옵션	Null 처리	키설정	설명
					~~'CREDIT_CARD', 'BANK_TRANSFER' 현재단계에선 필수 정보가 아님~~
amount	DECIMAL(10,2)		NOT NULL		결제 금액
status	VARCHAR(50)			NOT NULL	결제 상태 'PENDING', 'COMPLETED', 'FAILED' TO-DO >> 결제취소 상태도 정의하기
crea_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP			

- 실무상태
  - 제품,주문,결제 테이블들의 상태관리 연결이 어렵다!...
  - 상태관리 시나리오를 고민해보면 좋음
    - 각 상태에서 예상되는 동작들을 정리해봐야 한다.

4. 배송 관리

요구사항

- 판매자는 주문을 확인 후 배송을 시작할 수 있다.
- 구매자
  - 주문한 상품의 배송상태 확인이 가능해야 한다.
  - 배송상태에는 현재 진행상태, 배송업체, 예상 도착시간을 확인할 수 있다.
- 판매자/배송관리자
  - 판매자와 배송관리자는 배송 요청 목록을 확인할 수 있다.
  - 배송담당자는 배송 시작을 등록할 수 있다.
  - 배송이 완료되면 DELIVERED로 상태가 변경되어야 한다.
  -

- `shipments` 테이블

컬럼이름	데이터 타입	옵션	Null 처리	키설정	설명
id	BIGINT	AUTO_INCREMENT		PRIMARY KEY	
shipment_uuid	BINARY(16)		NOT NUL		배송 UUID
order_id	BIGINT		NOT NULL	FOREIGN KEY	주문 ID
tracking_number	VARCHAR(50)		NOT NULL	UNIQUE	배송 추적번호
carrier_name	VARCHAR(50)				운송 업체
status	VARCHAR(20)	DEFAULT 'PENDING'			배송 상태 'READY', 'SHIPPING', 'DELIVERED', 'FAILED','CANCELLED'

컬럼이름	데이터 타입	옵션	Null 처리	키설정	설명
					TO-DO >> 상태 시나리오 >> 바로구매후 배송, >> 당일배송 >> 그외 결제와 연관된 배송 시나리오
estimated_delivery_date	DATE				예상 배송 완료일 TO-DO >> 상품정보와 가까운 데이터 >> ex)직구상품
crea_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP			
updt_dt	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP			

-

1. 주문 기능 요구사항

[구매자 관점]

- 구매자는 장바구니에서 상품을 선택해 주문할 수 있다.
- 한 번의 주문에서 여러 개의 상품을 주문할 수 있다.
- 주문 시 배송 주소와 연락처를 입력한다.
- 주문 상태를 확인할 수 있어야 한다.

[판매자 관점]

- 판매자는 자신의 상품에 대해 주문이 들어온 내역을 확인할 수 있다.

[공통 기능]

- 주문은 **다수의 상품(OrderItem)**을 포함하며, 각 항목은 **상품 수량, 가격** 등의 정보를 가진다.
- 주문에는 **총액**, **주문일시**, **배송지 정보**, **상태**가 포함된다.
- 주문 상태는 다음과 같은 상태값을 가질 수 있다:
    - **주문 상태 플로우**

상태	설명	상태 전이 조건
`ORDERED`	주문 생성됨	유저가 주문 제출 시
`PAYMENT_PENDING`	결제 대기	외부 결제 연동 전 (optional 단계)
`PAID`	결제 완료	결제 성공 콜백
`INVENTORY_CONFIRMED`	재고 확인 완료	재고 확인 이벤트 후
`SHIPPED`	배송 시작	판매자/물류시스템에서 시작
`DELIVERING`	배송 중	택배사 정보 연동 중
`DELIVERED`	배송 완료	수령 확인 or 택배 API 응답
`CANCELLED`	전체 주문 취소	결제 전 또는 발송 전
`PARTIALLY_CANCELLED`	일부 아이템 취소	특정 아이템 취소 시

- 주문기능

✅ 기능 설계

기능	처리 방식	Kafka 적용 여부
주문 생성	동기 (POST)	❌
결제 처리	외부 연동 or 상태 저장	✅ 결제 완료 이벤트 발행
재고 차감	비동기 처리	✅ 결제 완료 이벤트 수신 후 재고 차감
배송 시작	동기 or Kafka trigger	✅ 재고 확인 후 배송 요청 이벤트
배송 완료	택배사 연동 or 유저 확인	✅ 배송 완료 이벤트 발행
주문 취소	동기 + 비동기 처리	✅ 취소 이벤트 → 결제 취소 & 재고 복원
주문 조회	상태 기반 정합성 조회	❌

- 주문 상태 라이프사이클

상태	설명	상태 전이 조건
`ORDERED`	주문 생성됨	구매자가 주문하면 생성됨
`PAID`	결제 완료	결제 처리 API 성공 시
`SHIPPED`	배송 시작됨	판매자 혹은 물류 처리로 배송 시작
`DELIVERING`	배송 중	택배사가 구매자에게 물건 배송중
`DELIVERED`	배송 완료	구매자가 등록한 주소에 택배 도착
`CANCELLED`	주문 취소됨	배송 전까지만 가능 주문취소는 주문전 취소, 결제 전 취소, 결제 완료후 반품접수가 있음 고민포인트들

-

✅ 주문 상태 플로우 개선

상태	설명	상태 전이 조건
`ORDERED`	주문 생성됨	유저가 주문 제출 시
`PAYMENT_PENDING`	결제 대기	외부 결제 연동 전 (optional 단계)
`PAID`	결제 완료	결제 성공 콜백
`INVENTORY_CONFIRMED`	재고 확인 완료	재고 확인 이벤트 후
`SHIPPED`	배송 시작	판매자/물류시스템에서 시작
`DELIVERING`	배송 중	택배사 정보 연동 중
`DELIVERED`	배송 완료	수령 확인 or 택배 API 응답
`CANCELLED`	전체 주문 취소	결제 전 또는 발송 전
`PARTIALLY_CANCELLED`	일부 아이템 취소	특정 아이템 취소 시

-
-

주문과 관련된 기능 간 영향도

1. 상품 기능과의 연관성

주문 상태	상품 영향
`ORDERED`	상품의 재고는 아직 보류 상태
`PAID`	**상품 재고 감소** (중요!)
`CANCELLED` (결제 전)	아무 영향 없음
`CANCELLED` (결제 후)	**상품 재고 복원** 필요
`DELIVERED`	사용자 리뷰 가능 등 기능 확장 가능

→ **재고는** `결제 시점` **에 감소**하는 게 일반적이며, 동시에 **재고 동기화** 트랜잭션을 고려해야 함.

---

2. 유저 기능과의 연관성

주문 상태	유저 기능 영향
`ORDERED`	구매자 활동 이력 저장 가능
`PAID`	유저의 구매금액 누적 (적립금/등급 제도 등 확장 가능)
`CANCELLED`	환불 또는 미결제 취소 처리
`DELIVERED`	리뷰 기능 활성화 / 알림 전송 등

→ JWT의 유저 UUID로 연관되며, 유저 활동 기반 분석 가능

주문 이벤트 정리

- 기본 플로우 : [ORDERED] → [PAID] → [SHIPPED]→ [SHIPPING] → [DELIVERED]
  - 주문생성
    - 케이스 1. 장바구니에 구매할 아이템들을 등록했을때
    - 케이스 2. 조회한 아이템을 보고 즉시주문할때
  - 결제 완료
    - 현재 프로젝트에선 결제기능의 상세한 요구사항과 기능들이 정의안됨
      (실제 이커머스 서비스 개발을 지향하지만, 현실적으로 적용을 못하니 상황과 수단은 흉내만 내기)
    - 결제기능을 대체하도록 테이블에 상태값으로 저장,관리
    - 주문기능과 배송기능에 상태 공유되어야 함
  - 배송 시작
    - 결제 완료 이벤트가 발생하면 주문 조회 기능 배송기능 내부로 이벤트 공유되어야 함
    - 택배사에 배송요청 정보가 전달되어야 함
    - 현재 프로젝트에선 실제 택배서비스 업체와의 요청정보 통신이 안되니 테이블 상태값이나 미들웨어로 대체
      (실제 이커머스 서비스 개발을 지향하지만, 현실적으로 적용 못하니 상황과 수단은 흉내만 내기)
  - 배송 완료
    - 배송 완료 이벤트는 주문조회, 배송기능 내부에 전달되어야 함
    - 배송 완료기준 정의 필요
      - 택배사에서 도착상황을 서비스에 전달
      - 유저가 직접 수령했을때
- 주문취소 : [ORDERED] → [CANCELLED]
- 취소에 대한 시나리오들 정리
  - 1.주문 취소
    - 유저가 취소할때

            - 최종 결제 전 취소 가능
            - 최종 결제 완료 후부터 택배사 배송 시작 상태 전까지 취소가능
        - 시스템에서 취소 : 재고가 0 일때
            - 재고에 대한 케이스들
            1. 판매자가 재고를 0으로 바꾼경우
            2. 판매가 완료될때 자동차감 = 결제가 끝났을때.
    - 2.결제취소
        - 구체적으로 언제?
            - 최종 결제 완료 후 결제취소가 가능해야 한다
        - 결제가 완료된 후 반품요청은?
            - 최종 결제 완료 후, 바로 반품 접수는 못하게 막는다
            - 반품 접수는 구매자에게 물품이 배송완료된 시점부터 가능하다.
    - 3.배송취소
        - 결제가 취소되면 누가 어떻게 취소 정보를 전달하는가??
            - 결제 취소는 구매자의 결제 정보 오류, 구매금액보다 적은 잔액부족으로 발생한다
            - 결제 기능내부에서 취소요청을 받으면 주문기능과 배송기능으로 전파한다
            -
    - 4.부분취소
        - 주문안의 여러 아이템 중,
        - 부분 취소가 가능한 상황이 있을텐데?
        - 전체 취소가 가능할텐데?
            - 결제잔액의 문제가 잇을건데
- 더 생각하기
    - 각 케이스에서 상품의 재고가 변경되는 방식
    - 각 케이스별로 상태를 전파 시키는 방법
    - 주문 + 결제 + 상품정보 >> 트랜잭션으로 묶어서 처리해야 할 단계.
    - 메세지큐를 사용한다면, 플로우가 변할것이다
    - 케이스에 맞게 적절하게 사용하는게 적합하다
    - 대용량 트래픽? 별거아니다. 현재 프로젝트는 서버 스케일링으로도 극복가능하다. (scalable architecture)
    - 기능이 다 완료되고(플로우) 그후에 검토해도된다
    - 원래 목표는 대용량 트래픽을 일으켜서 서비스 개선하기인데, 카프카를 도입할 만큼 기능들이 이벤트기반이며, 이벤트처리가 빨라야한다.
        - B2C 계열 인기있는 서비스들은 카프카와같은 미들웨어를 활용한다 > 이 경험이 필요하다

주문 기능 요구사항 (이벤트 기반 확장)

[구매자 관점]

- 구매자는 장바구니에서 상품을 선택해 주문할 수 있다.
- 한 번의 주문에서 여러 개의 상품을 주문할 수 있다 (`OrderItem` 리스트).
- 주문 시 배송 주소, 연락처를 입력한다.

- 주문 상세 내역과 상태(ORDERED, PAID, SHIPPED, DELIVERED, CANCELLED)를 확인할 수 있다.

Kafka 활용 가능

- `order.created` 이벤트 발행 → 배송, 알림, 재고, 포인트 서비스에 비동기 전달

[판매자 관점]

- 판매자는 자신의 상품에 대한 주문 내역을 확인할 수 있다.
- 주문 상태 변경(예: 배송 시작)은 판매자가 직접 수행하거나, 물류 시스템 연동으로 변경된다.

Kafka 활용 가능

- `order.shipped` 이벤트 발행 → 알림, 유저 리뷰 유도, 구매 확정 이벤트 트리거

[공통 기능]

항목	설명
Order	구매자, 배송지, 연락처, 총금액, 주문일시, 상태 포함
OrderItem	상품, 수량, 단가 포함
상태 변경 흐름	`ORDERED` → `PAID` → `SHIPPED` → `DELIVERED` / `CANCELLED`
상태 값 관리	enum으로 정의된 `OrderStatus`

Kafka 기반 이벤트 흐름

주요 이벤트 & Kafka Topic

Kafka Topic	이벤트 발생 시점	이벤트 내용
`order.created`	주문 생성 시	주문번호, 구매자ID, 상품리스트, 수량, 가격, 주소
`order.paid`	결제 완료 시	주문번호, 결제금액, 결제일시
`order.shipped`	배송 시작 시	주문번호, 송장번호, 택배사
`order.delivered`	배송 완료 시	주문번호, 수령확인 시각
`order.cancelled`	주문 취소 시	주문번호, 취소사유

✅ Kafka 소비자 서비스 예시

1. 📉 InventoryService

- `order.created` → 재고 차감
- `order.cancelled` → 재고 복원

2. 📬 NotificationService

- 모든 상태 변화 이벤트 소비 → 푸시/이메일 발송
  - `order.paid` → 결제 확인 알림
  - `order.shipped` → 배송 출발 알림

3. 🪙 RewardService

- `order.paid` → 포인트 적립 or 쿠폰 발급
- `order.delivered` → 리뷰 요청 알림 트리거

4. 📊 AnalyticsService

- 모든 주문 이벤트 수집 → 실시간 주문 통계 분석, BI 리포트 생성

✅ 주문 상태 전이 설계 (이벤트 기반)

```
plaintext
복사편집
[ORDERED]
  └ Kafka → "order.created" → InventoryService, NotificationService

[PAID]
  └ Kafka → "order.paid" → RewardService, ShippingService

[SHIPPED]
  └ Kafka → "order.shipped" → NotificationService

[DELIVERED]
  └ Kafka → "order.delivered" → NotificationService, RewardService

[CANCELLED]
  └ Kafka → "order.cancelled" → InventoryService, NotificationService
```

도메인 서비스 책임 구분

서비스	책임
OrderService	주문 생성/조회/취소 (Kafka Producer)
PaymentService (모의)	결제 완료 이벤트 트리거
InventoryService	Kafka Consumer → 재고 차감
ShippingService	Kafka Consumer → 배송 준비 처리
NotificationService	Kafka Consumer → 알림 발송
RewardService	Kafka Consumer → 포인트 적립 / 리뷰 요청
AnalyticsService	Kafka Consumer → 통계 수집

✅ 마무리 요약

- 주문 생성, 결제, 배송, 취소는 각기 다른 서비스가 관여할 수 있는 **분산 이벤트 기반 처리 흐름**이다.
- Kafka는 이 흐름에서 **비동기 데이터 전달**, **장애 격리**, **확장성 확보**에 결정적인 역할을 한다.
- 또한 **이벤트 로그 자체가 시스템의 상태 이력을 보존**하는 장점도 있다.

주문 상태 기반 기능 시나리오 정리

✅ 1. 핵심 기능 구조 및 책임

기능	책임	이벤트 연동	설명
상품 기능	상품 등록, 재고 변경, 조회	`상품 생성`, `재고 차감`	재고 차감은 결제 완료 후
주문 기능	주문 생성, 상태 관리, 주문 상세 조회	`주문 생성`, `결제 요청`, `결제 완료`, `재고 확인`, `배송 요청`	도메인 이벤트 발행 책임
결제 기능 (Mock)	결제 요청 처리, 결제 성공/실패 처리	`결제 성공`, `결제 실패`	외부 연동 없음, 시뮬레이션 처리
배송 기능 (Mock)	배송 등록, 상태 변경	`배송 요청`, `배송 완료`, `배송 실패`	상태 변경은 이벤트 기반

✅ 2. 주문 상태 기반 시나리오 유형

시나리오	상태값 흐름	상태 변화	Kafka 이벤트 흐름	비고
정상 주문	ORDERED → PAID → INVENTORY_CONFIRMED → SHIPPED → DELIVERED	모든 정상 상태	`order.created`, `payment.success`, `inventory.confirmed`, `delivery.started`, `delivery.completed`	정상 흐름
주문 후 취소	ORDERED → CANCELLED	`order.cancelled`	배송 전 가능	
결제 후 주문 취소	PAID → CANCELLED	`order.cancelled`, `delivery.cancelled`	배송 시작 전까지 가능	
결제 실패	ORDERED → PAYMENT_PENDING → CANCELLED	`payment.failed`, `order.cancelled`	예: 잔액 부족	
재고 부족	ORDERED → PAYMENT_PENDING → INVENTORY_FAILED → CANCELLED	`inventory.failed`, `order.cancelled`	상품 기능이 재고 거절	

✅ 3. 주요 Kafka 이벤트 설계

이벤트명	발행 주체	수신 주체	설명
`order.created`	주문	결제, 상품	주문 생성 시 발행
`payment.requested`	주문	결제	결제 요청 시
`payment.success`	결제	주문	결제 성공 콜백
`payment.failed`	결제	주문	결제 실패 시
`inventory.confirmed`	상품	주문	재고 확보 완료
`inventory.failed`	상품	주문	재고 부족 시 취소
`delivery.requested`	주문	배송	배송 시작 요청
`delivery.completed`	배송	주문	배송 완료 수신
`delivery.failed`	배송	주문	배송 실패 처리

✅ 4. 결제/배송 기능 시뮬레이션 설계

✅ 결제 기능

- `payment.requested` 수신
- 90% 확률로 `payment.success` 발행
- 10% 확률로 `payment.failed` 발행

✅ 배송 기능

- `delivery.requested` 수신
- 3초 후 `delivery.completed` 발행

✅ 5. 주문 기능 상세 책임

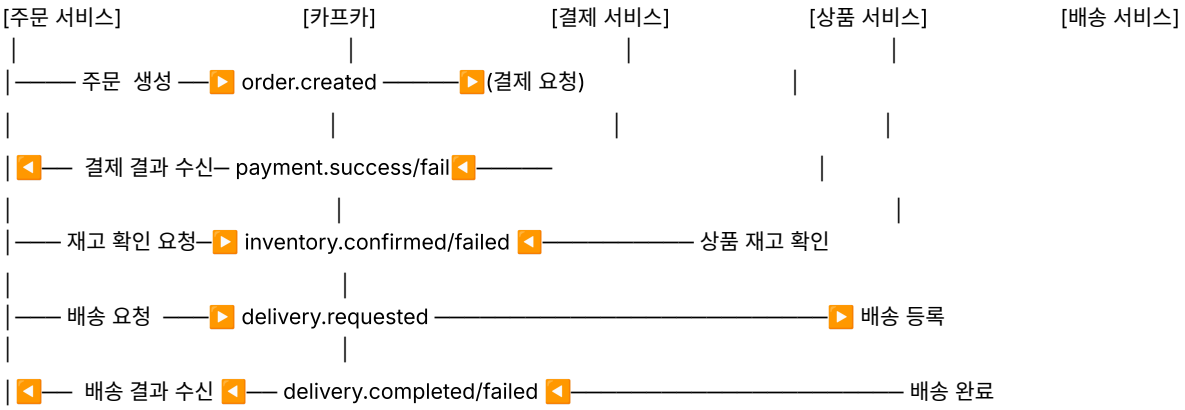
단계	기능 책임	이벤트
주문 생성	주문 저장 + 이벤트 발행	`order.created`
결제 요청	이벤트 발행	`payment.requested`
결제 완료 수신	주문 상태 변경	`payment.success`
재고 확인 요청	이벤트 발행	`inventory.confirmed` or `inventory.failed`
배송 요청	이벤트 발행	`delivery.requested`
배송 완료 수신	주문 상태 완료	`delivery.completed`

✅ 6. 트랜잭션과 이벤트 기반 처리 비교

항목	동기 트랜잭션 방식	비동기 이벤트 기반

재고 차감	주문-상품 DB 트랜잭션	`payment.success` 수신 후 `inventory.confirmed`
결제 요청	주문 생성 직후 처리	`order.created` 후 이벤트 기반 요청
배송 요청	결제 성공 후 즉시	재고 확인 후 `delivery.requested`

✅ 아키텍처 다이어그램



EDA 기반 이커머스 서비스 개발

직렬화

트러블 슈팅

설계 고민

모던 웹 아키텍처와 패턴 체계 정리

아키텍처 분류 체계와 레벨별 정리

EDA 아키텍쳐 설계 고민포인트 정리